NutriTrackWeb

Documentação Técnica

NutriTrackWeb – Aplicação web para nutrição

Nome	Stefany Castelão dos Santos
Turma	3. IT INF – Labor
Grupo	3APP1
Data	08.11.2026
Professor	Herr Platzer
Projeto	NutriTrackWeb

Índice

1 Visão geral e arquitetura 3
1.1 Componentes 3
1.2 Fluxo de uma requisição 3
1.3 Papéis 4
2 Glossário 4
3 Estrutura do projeto 5
3.1 Arquivos e pastas 5
3.2 CSS e JavaScript 6
4 Banco de dados 6
4.1 Tabelas 7
4.2 Terceira forma normal 8
4.3 Códigos de status 8
5 Segurança e validação de dados 9
5.1 Medidas em resumo 9
5.2 Validação de dados 10
5.3 Uploads e pastas protegidas 10
5.4 Registros de teste para a proteção contra XSS 10
6 Instalação e início 11
6.1 Passos 11
6.2 Configuração no php.ini 12
6.3 HTTPS local (mkcert) 12
Certificado 13
6.4 Contas de demonstração 13
7 Limites e extensões 14
7.1 Limites conhecidos da versão 1 14
7.2 Extensões futuras 14

1 Visão geral e arquitetura

O NutriTrackWeb é uma aplicação web em PHP em que uma nutricionista gerencia seus clientes. As páginas PHP falam com um banco de dados MariaDB através do PDO e entregam HTML pronto ao navegador. O significado dos termos técnicos está no capítulo 2.

1.1 Componentes

Componente	Utilizado	Função
Linguagem	PHP, sem framework (um arquivo por página)	Lógica e saída das páginas
Banco de dados	MariaDB, banco *mydb*	Guarda usuários, clientes, consultas, mensagens, pagamentos etc.
Acesso aos dados	PDO com Prepared Statements	Acesso seguro ao banco de dados
Servidor web	Apache (XAMPP, Windows) com HTTPS	Entrega as páginas
Interface	HTML, CSS, um pouco de JavaScript	Exibição; cores e logo podem ser alterados nas configurações
Idiomas	Inglês, alemão, português	Os textos ficam no array *$t* de cada arquivo PHP

1.2 Fluxo de uma requisição

1. O navegador abre uma página PHP, por exemplo *client_home.php*.
2. A página carrega o *config.php*: ali a sessão é iniciada (timeout de 30 minutos) e a conexão *$pdo* com o banco é criada.
3. A página verifica se há alguém logado e qual é o papel (*users.role*: 0 = User, 1 = Admin). Sem o papel correto, volta para o login.
4. Em formulários (POST), primeiro o token CSRF é verificado e depois os dados são validados.
5. A página lê ou grava dados com Prepared Statements. Gravações que dependem umas das outras rodam em uma transação (em caso de erro, *rollBack*).
6. Depois de salvar, a página redireciona (padrão PRG), para que recarregar a página não grave nada em duplicidade.
7. Todo texto exibido passa por *e()* (= *htmlspecialchars*), para que nenhum código externo rode no navegador.

1.3 Papéis

Papel	Pode	Não pode
Admin (nutricionista)	Revisar registros e gerenciar clientes, consultas, documentos, fotos de refeições, mensagens, pagamentos e configurações	–
User (cliente)	Ver e editar os próprios dados, consultas, documentos, fotos de refeições, mensagens e pagamentos	Abrir páginas de Admin; ver dados ou arquivos de outras pessoas

Os clientes se registram sozinhos. O acesso só vale depois da aprovação pela Admin (*clients.approval_status*: 0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos).

2 Glossário

Os termos mais importantes, explicados de forma simples:

Termo	Explicação simples
Servidor web (Apache)	O programa que entrega as páginas quando o navegador pede. Como um garçom que recebe o pedido e traz a comida.
PHP	Uma linguagem de programação que roda no servidor. O PHP monta a página e o navegador recebe apenas o HTML pronto.
XAMPP	Um pacote para o próprio computador com Apache, MariaDB, PHP e phpMyAdmin em uma única instalação.
MariaDB	O servidor de banco de dados: o “arquivo” onde todos os dados ficam guardados em tabelas. É uma evolução livre do MySQL; o “MySQL” do XAMPP na verdade é MariaDB.
SQL	A linguagem usada para ler e gravar dados no banco (por exemplo *SELECT*, *INSERT*, *UPDATE*).
PDO	PHP Data Objects: a forma padrão de o PHP conversar com um banco de dados. Como uma tomada única que oferece o caminho seguro com Prepared Statements.
Prepared Statement	Um comando SQL com marcadores (*?*). Os valores são enviados separadamente. Como um formulário com lacunas: o que se escreve na lacuna não consegue mudar o texto do formulário.
Session (sessão)	Uma anotação no servidor para cada visitante. O navegador guarda só um número (como a ficha do guarda-volumes); os dados ficam no servidor.

Termo	Explicação simples
Hash	Uma espécie de impressão digital de uma senha. Dá para calculá-la, mas não dá para voltar dela à senha.
CSRF	Um site externo tenta enviar, em seu nome, um formulário para o nosso site. Proteção: um token secreto no formulário.
XSS	Alguém escreve código (por exemplo <*script*>) em um campo de texto para que rode no navegador de outras pessoas. Proteção: as saídas são exibidas como texto puro.
SQL Injection	Alguém escreve comandos SQL em um campo para que o banco os execute. Proteção: Prepared Statements.
Transação	Um grupo de gravações que só valem em conjunto: tudo ou nada.
.htaccess	Um pequeno arquivo de configuração do Apache. Aqui ele bloqueia o acesso direto a uploads confidenciais e impede código de programa nas pastas de upload.
HTTPS e certificado	O HTTPS criptografa a conexão entre navegador e servidor. O certificado é o “documento de identidade” do servidor, para o navegador confiar nele.
mkcert	Uma ferramenta que cria um certificado para o próprio computador, em que o navegador deste computador confia.
phpMyAdmin	Uma interface web para ver, importar e exportar o banco de dados.

3 Estrutura do projeto

3.1 Arquivos e pastas

O projeto fica em *C:\xampp\htdocs\nutritrackweb*. Todos os 24 arquivos PHP estão na pasta principal. Além deles há a pasta *icons/* (símbolos SVG) e a pasta *uploads/* (arquivos enviados).

Área	Arquivos	Função
Base	*config.php*	Início da sessão, timeout, conexão com o banco (PDO)
Acesso	*index.php*, *login.php*, *logout.php*, *register.php*	Página inicial, login, logout, auto-registro

Área	Arquivos	Função
Admin	*admin_dashboard.php*, *registration_review.php*, *clients.php*, *appointments.php*, *documents.php*, *meal_photos.php*, *messages.php*, *payments.php*, *settings.php*	Visão geral, revisar registros, clientes, consultas, documentos, fotos de refeições, mensagens, pagamentos, configurações
User	*client_home.php*, *client_status.php*, *client_profile.php*, *client_appointments.php*, *client_documents.php*, *client_meal_photos.php*, *client_messages.php*, *client_payments.php*	Início, status da aprovação, perfil, consultas, documentos, fotos de refeições, mensagens, pagamentos
Acesso a arquivos	*download.php*, *view_image.php*	Único caminho para os arquivos enviados, com verificação de permissão
Pastas	*icons/*, *uploads/documents/*, *uploads/meal_photos/*, *uploads/profile/*	Símbolos; pastas de upload protegidas

Todas as páginas têm a mesma estrutura: em cima a lógica PHP (verificar papel, processar POST, carregar dados) e embaixo o HTML. Os textos em inglês, alemão e português ficam no início de cada arquivo, no array *$t*. O código-fonte está comentado em alemão do início ao fim.

3.2 CSS e JavaScript

CSS e JavaScript não ficam em arquivos próprios, e sim dentro de cada página PHP: o CSS em um bloco <*style*> no início do arquivo e o JavaScript em um bloco <*script*> no final. Para o CSS há um motivo: as cores vêm do banco de dados (tabela *nutritionist_profile*). O PHP as escreve no bloco como variáveis CSS (*--primary*, *--accent* etc.) quando a página é carregada. Assim, a aparência de toda a aplicação muda assim que a Admin escolhe outras cores nas configurações.

> **Nota:** Arquivos próprios (*style.css*, *app.js*) seriam a solução mais profissional, porque regras iguais ficariam em um só lugar e o navegador poderia guardar os arquivos em cache. Isso está planejado para uma versão futura (ver capítulo 7).

4 Banco de dados

O banco de dados *mydb* tem 10 tabelas (InnoDB, utf8) e atende à terceira forma normal. A exportação completa está em *nutritrackweb_database_dump.sql*.

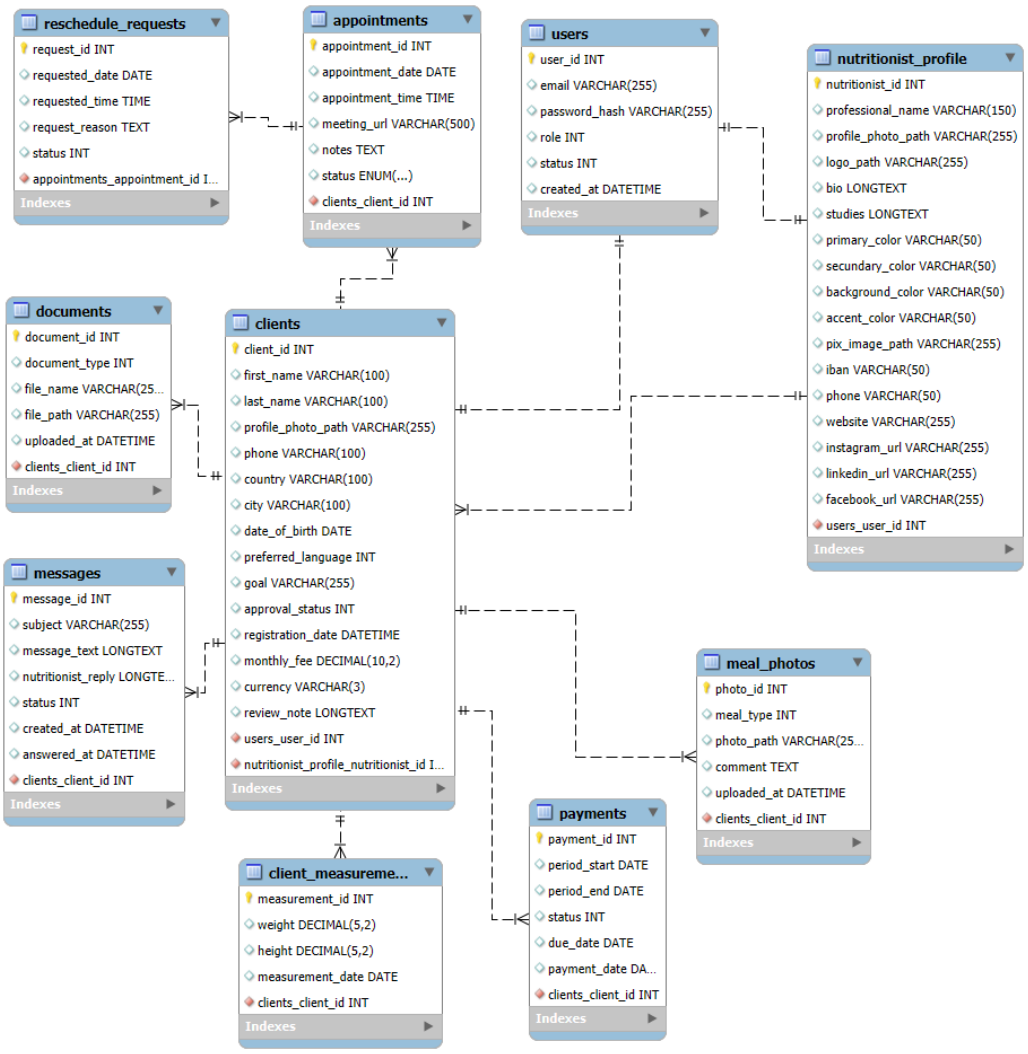


Diagrama ER do banco de dados mydb (MySQL Workbench)

4.1 Tabelas

Tabela	Conteúdo	Ligação (chave estrangeira)
users	Login: e-mail, *password_hash*, papel, status	–
nutritionist_profile	Nome, bio, cores, logo, IBAN, links de contato da nutricionista	*users_user_id*
clients	Dados cadastrais, objetivo, status de aprovação, valor mensal, moeda	*users_user_id*, *nutritionist_profile_nutritionist_id*
client_measurements	Peso, altura, data	*clients_client_id*
appointments	Data, horário, link da reunião, observações, status	*clients_client_id*

Tabela	Conteúdo	Ligação (chave estrangeira)
reschedule_requests	Remarcação: data desejada, motivo, status	*appointments_appointment_id*
documents	Arquivos PDF (nome, caminho, tipo, data)	*clients_client_id*
meal_photos	Fotos de refeições com comentário	*clients_client_id*
messages	Mensagem do cliente e resposta da nutricionista	*clients_client_id*
payments	Período de cobrança, vencimento, data do pagamento, status	*clients_client_id*

4.2 Terceira forma normal

- Cada tabela tem exatamente uma chave primária de uma coluna (por exemplo *client_id*).
- Cada informação aparece uma só vez: o valor de um pagamento não é guardado duas vezes, ele vem de *clients.monthly_fee*.
- País e cidade são campos de texto livre e não tabelas próprias, porque na aplicação só são exibidos.
- Os valores de status são códigos numéricos, explicados no código PHP (ver 4.3).

Leitura e gravação passam sempre pelo PDO em todas as páginas. Gravações em várias tabelas (por exemplo o registro com *users*, *clients* e exame de sangue em *documents*) acontecem em uma transação: ou tudo é salvo ou nada.

4.3 Códigos de status

Campo	Valores
users.role	0 User, 1 Admin
clients.approval_status	0 Pending, 1 Approved, 2 Rejected, 3 MoreInfos
messages.status	0 Nova, 1 Respondida, 2 Encerrada
payments.status	0 Open, 1 Paid, 2 Cancelled
reschedule_requests.status	0 Pending, 1 Approved, 2 Rejected
appointments.status	scheduled, rescheduled, completed, cancelled (texto)

5 Segurança e validação de dados

5.1 Medidas em resumo

Risco	Medida	Explicação simples
Senhas legíveis	*password_hash()* ao salvar, *password_verify()* no login (*register.php*, *login.php*). Só o hash fica em *users.password_hash*. Mínimo de 8 caracteres.	A senha nunca é guardada, só uma “impressão digital”. No login, compara-se a impressão digital do que foi digitado. Se o banco for roubado, as senhas continuam ilegíveis.
Sessão roubada	*session_regenerate_id(true)* depois do login. Após 30 minutos sem atividade, *session_unset()* e *session_destroy()* (todas as páginas).	Depois do login existe uma nova “ficha de guarda-volumes”; a ficha antiga não vale mais nada. Após 30 minutos de pausa, a ficha perde a validade.
Formulários falsificados (CSRF)	Token aleatório (*random_bytes*) na sessão e no formulário, comparado com *hash_equals()* (todas as páginas com POST).	Cada formulário contém um token secreto que só o nosso site conhece. Se não bater, o formulário é recusado.
Código externo na saída (XSS)	Toda saída passa por *e()* = *htmlspecialchars()* (todas as páginas).	Quem escreve <*script*> em um campo vê depois apenas texto normal. Não roda como programa.
SQL Injection	Somente Prepared Statements com marcadores *?* (todas as páginas).	O comando é enviado primeiro como modelo e os valores depois, separados. O banco trata o que foi digitado só como dados, nunca como comando.
Papel errado, dados de outras pessoas	Cada página verifica *users.role*. Users só veem linhas com o próprio *client_id*.	Um “porteiro” em cada página: um User não chega às páginas de Admin nem aos dados de outras pessoas.
Dados salvos pela metade	Transação com *rollBack* (*register.php*, *appointments.php*, *client_status.php*, *registration_review.php*).	Tudo ou nada: em caso de erro, tudo é desfeito.
Envio duplicado	Redirecionamento depois de salvar (padrão PRG, todos os formulários).	Depois de salvar, a página recarrega. F5 não envia o formulário de novo.
Validação do navegador contornada	Validação também no PHP, não só com *required* no navegador.	A validação do navegador pode ser contornada. Por isso o servidor valida também.

Risco	Medida	Explicação simples
Conexão interceptada	HTTPS com certificado (ver 6.3).	Os dados entre navegador e servidor são criptografados.

5.2 Validação de dados

As validações sempre rodam no servidor; no navegador existem apenas adicionalmente (por exemplo *required*). Exemplos de *register.php*:

- Campos obrigatórios não podem ficar vazios.
- E-mail: *filter_var(..., FILTER_VALIDATE_EMAIL)* e no máximo 255 caracteres.
- Senha: no mínimo 8 caracteres.
- Objetivo: no máximo 255 caracteres.
- Política de privacidade: a caixa de seleção também é verificada no PHP (*isset($_POST['privacy'])*).

Os erros aparecem como mensagem no idioma escolhido. Os valores já digitados permanecem no formulário.

5.3 Uploads e pastas protegidas

- As pastas *uploads/documents* e *uploads/meal_photos* contêm um *.htaccess* com *Require all denied*. O navegador não consegue abrir nenhum arquivo diretamente ali.
- A pasta *uploads/profile* (fotos de perfil, logo, QR code PIX) tem um *.htaccess* que bloqueia arquivos de programa (por exemplo PHP) e oculta a lista de arquivos. As imagens em si são exibidas diretamente, porque não são confidenciais (por exemplo, a foto na página de login).
- Os arquivos só são entregues por *download.php* e *view_image.php*. Ambos verificam login e permissão. Para arquivos de outras pessoas ou inexistentes, a resposta é sempre a mesma, *404*, para que ninguém descubra se um arquivo existe.
- Os nomes dos arquivos são aleatórios. É entregue apenas o *basename()* dentro da pasta fixa (*realpath*).
- O conteúdo é verificado, não só a extensão: um PDF começa com *%PDF-* (no máximo 10 MB); imagens são verificadas com *getimagesize()* (no máximo 8 MB).

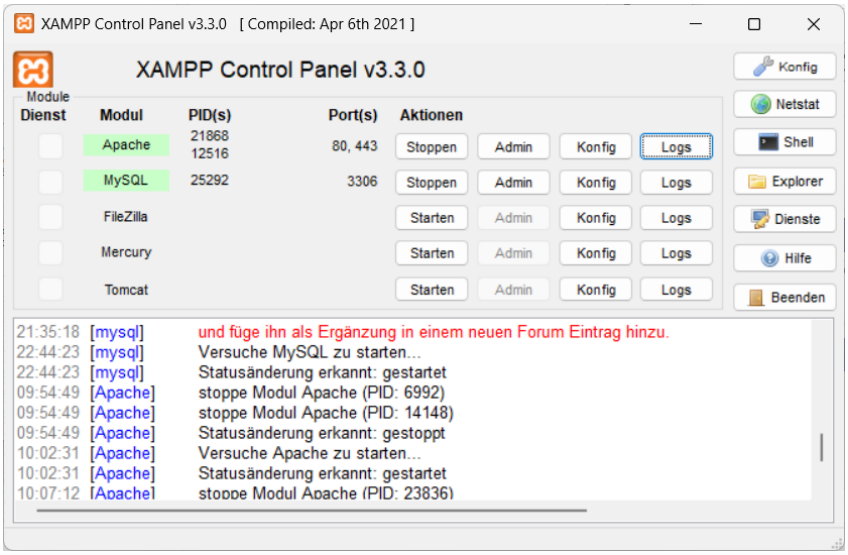
5.4 Registros de teste para a proteção contra XSS

No banco de demonstração há, de propósito, um cliente com o nome *Teste* e uma mensagem com *<script>alert(1)</script>*. A aplicação exibe esse texto sem alterá-lo e sem executá-lo. Isso comprova que a proteção contra XSS (*e()*) funciona.

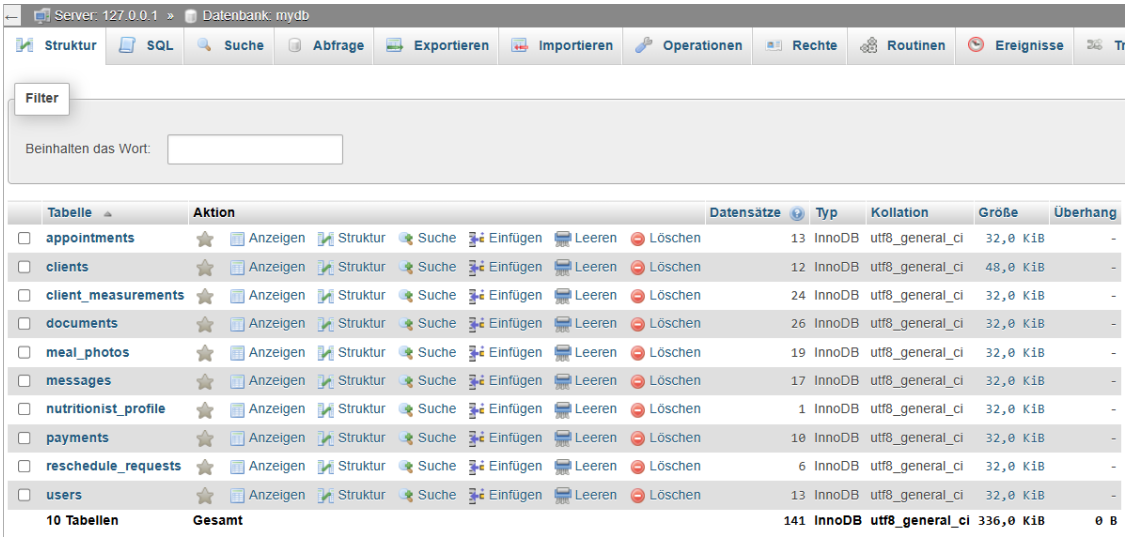
6 Instalação e início

6.1 Passos

1. Instalar o XAMPP e, no Control Panel, iniciar **Apache** e **MySQL** (MariaDB).
2. Copiar a pasta do projeto para *C:\xampp\htdocs\nutritrackweb*\. A pasta *uploads/* precisa ter as subpastas *documents*, *meal_photos* e *profile*. Em cada uma há um arquivo *.htaccess*.
3. No phpMyAdmin (*http://localhost/phpmyadmin*), criar o banco *mydb* e importar *nutritrackweb_database_dump.sql*.
4. Conferir no *config.php*: banco *mydb*, usuário *castelao*, senha vazia (somente para desenvolvimento local).
5. Abrir *https://localhost/nutritrackweb/* no navegador e entrar com uma conta de demonstração (ver 6.4).



Painel de controle do XAMPP com Apache e MySQL em execução



phpMyAdmin: banco mydb com as 10 tabelas

6.2 Configuração no php.ini

upload_max_filesize e *post_max_size* precisam ser pelo menos *10M*, senão o envio de PDFs falha. Nesta instalação do XAMPP ambos estão em *40M*. Depois de uma alteração, reiniciar o Apache.

6.3 HTTPS local (mkcert)

Para que *https://localhost* funcione sem aviso do navegador, a ferramenta mkcert gera um certificado confiável localmente.

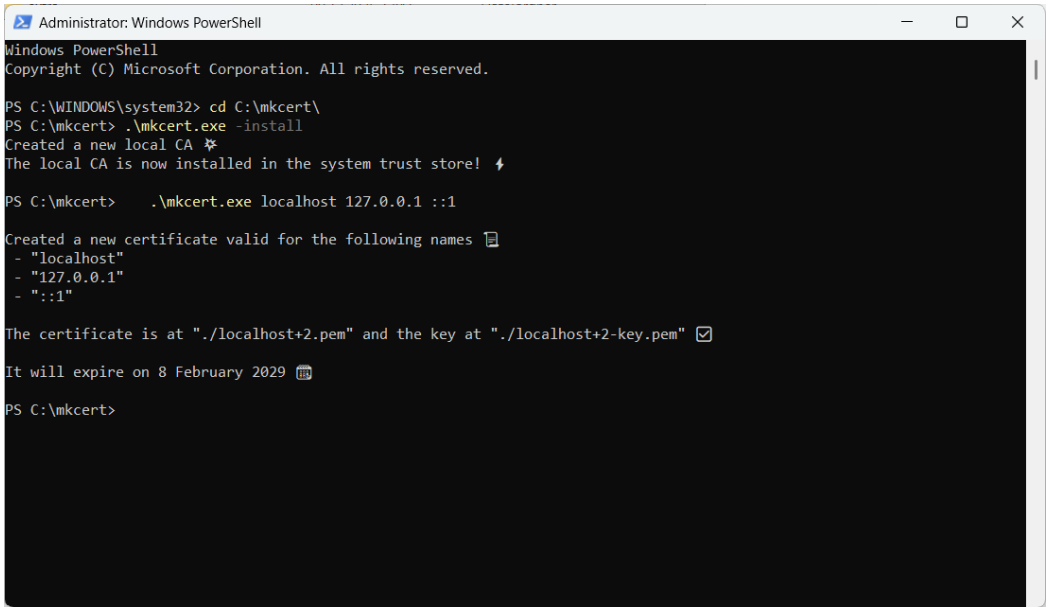
1. Baixar o mkcert na página do projeto no GitHub (página “Releases”, arquivo para Windows, *mkcert-…-windows-amd64.exe*). Colocar o arquivo na pasta *C:\mkcert* e renomeá-lo para *mkcert.exe*.
2. Abrir o **Windows PowerShell como administrador** e instalar a autoridade certificadora local:

```
cd C:\mkcert\
.\mkcert.exe -install
```

3. Gerar o certificado para *localhost*:

```
.\mkcert.exe localhost 127.0.0.1 ::1
```

O mkcert cria os arquivos *localhost+2.pem* (certificado) e *localhost+2-key.pem* (chave).

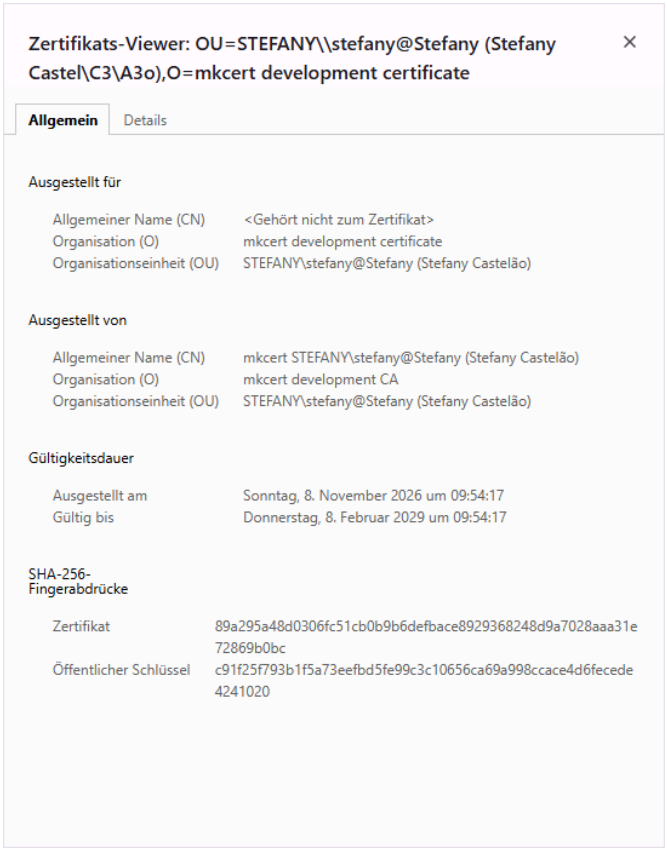


PowerShell: instalar a autoridade certificadora local e gerar o certificado

4. Copiar os dois arquivos para a pasta do Apache no XAMPP e informá-los em *C:\xampp\apache\conf\extra\httpd-ssl.conf*, em *SSLCertificateFile* e *SSLCertificateKeyFile*.
5. Reiniciar o Apache no XAMPP Control Panel e abrir *https://localhost/nutritrackweb/*.

Certificado

Informação	Valor
Emitido por	mkcert development CA
Emitido para	mkcert development certificate (nomes: *localhost*, *127.0.0.1*, *::1*)
Válido desde	08.11.2026, 09:54
Válido até	08.02.2029, 09:54
Arquivos	*localhost+2.pem*, *localhost+2-key.pem*



Visualizador de certificado do navegador com o período de validade

> **Importante:** O certificado só é confiável no computador em que *mkcert -install* foi executado. Em outro computador o navegador mostra um aviso. Por isso a apresentação é feita no notebook próprio.

6.4 Contas de demonstração

A exportação do banco contém contas de demonstração com dados fictícios. As senhas estão no arquivo *Zugangsdaten.txt*, no ZIP da entrega. O status de uma conta (aprovada, pendente, recusada) pode mudar durante os testes.

Nome	E-mail	Papel
Marianne Böhm, MSc	admin-demo@ntw.at	Admin
Lucas Pereira	user1-demo@ntw.at	User
Anna Fischer	user2-demo@ntw.at	User
Emily Carter	user3-demo@ntw.at	User
Lukas Schneider	user4-demo@ntw.at	User
Beatriz Oliveira	user5-demo@ntw.at	User
Leonardo Barbacoa	barbacoa@gmx.at	User
Roger Batista	user7-demo@ntw.at	User
Amanda Moura	user8-demo@ntw.at	User
Sophie Gruber	user9-demo@ntw.at	User
Markus Steiner	user10-demo@ntw.at	User
Teste (teste de XSS)	user11-demo@ntw.at	User
Amanda Silva	user12-demo@ntw.at	User

7 Limites e extensões

7.1 Limites conhecidos da versão 1

- A aplicação roda apenas localmente. A hospedagem na internet não foi implementada, embora exista um ticket para isso (ver 7.2).
- O acesso ao banco não tem senha, e o HTTPS funciona só com o certificado local do mkcert. Isso serve para desenvolvimento e apresentação, não para uso real.
- CSS e JavaScript ficam em cada arquivo PHP separadamente (ver 3.2). País e cidade são textos livres, e os textos da interface ficam em cada arquivo em vez de arquivos de idioma centrais.
- Clientes não podem ser desativados e reativados (código não utilizado foi removido).
- Não há testes automáticos. Os testes foram manuais, com um roteiro de 132 pontos.

7.2 Extensões futuras

- **Hospedagem:** publicação na internet com domínio próprio, certificado real (Let's Encrypt) e um usuário do banco com senha forte.

- Mover **CSS e JavaScript** para arquivos próprios (*style.css*, *app.js*); só as variáveis de cor continuam dinâmicas.
- Arquivos de idioma centrais em vez de textos em cada arquivo PHP.
- Esqueci a senha (redefinição por e-mail).
- Aviso de contraste ruim nas configurações de cores.
- Revisão formal de segurança de toda a aplicação.